

Flutter : Persistance des Données

Module	Développement Mobile avec Flutter
Niveau	IAM1 — 2025/2026
Durée totale	TP1 : 3h TP2 (TaskVault) : 3h30
Solutions étudiées	shared_preferences · SQLite (sqflite) · Fichiers (path_provider)

Ces deux TP ont le **même sens et sont complémentaires** : le TP1 introduit chaque solution de persistance de manière isolée (exercices séparés), tandis que le TP2 les intègre toutes les trois dans une application Flutter complète et cohérente appelée **TaskVault**. Le présent compte rendu synthétise la théorie, les implémentations et le code source de l'application.

1. PARTIE THÉORIQUE — Les 3 Solutions de Persistance

1.1 Tableau comparatif

Flutter propose trois approches locales de persistance, chacune adaptée à un cas d'usage précis :

Solution	Package(s)	Adapté pour	À éviter si
shared_preferences	shared_preferences	Préférences, tokens, flags, petits paramètres	Listes longues, objets complexes
Fichiers (dart:io)	path_provider + dart:io	Logs, cache, JSON, exports, fichiers binaires	Données structurées avec recherche/tri
SQLite	sqflite + path	Données structurées, relations, requêtes complexes	Données très simples, Web (non supporté)

1.2 Règle de décision

1 à 10 valeurs simples (bool/int/String)	→ shared_preferences
Fichiers texte, JSON, images, cache	→ path_provider + dart:io
Données tabulaires avec recherche/tri/filtres	→ sqflite
Synchronisation cloud / multi-utilisateurs	→ Firebase Firestore

2. TP1 — Exercices Isolés par Solution

2.1 Partie A — shared_preferences

Le package **shared_preferences** stocke des paires clé-valeur simples sur le disque. Il utilise UserDefaults (iOS), SharedPreferences (Android) et localStorage (Web).

Types supportés et méthodes :

Type Dart	Setter	Getter
int	setInt('cle', val)	getInt('cle') → int?
double	setDouble('cle', val)	getDouble('cle') → double?
bool	setBool('cle', val)	getBool('cle') → bool?
String	setString('cle', val)	getString('cle') → String?
List<String>	setStringList('cle', val)	getStringList('cle') → List<String>?

■ **Comportement clé** : Les setters font deux opérations séquentielles : (1) mise à jour synchrone en mémoire, (2) persistance asynchrone sur disque. Le getter lit toujours depuis la mémoire. Toujours utiliser ?? pour les valeurs par défaut car le getter retourne null si la clé n'existe pas.

Exemple — Compteur persistant :

Dans `initState()`, charger avec `prefs.getInt('counter') ?? 0`, puis appeler `setState()`. À chaque incrément, sauvegarder avec `prefs.setInt('counter', _counter)`.

2.2 Partie B — path_provider + dart:io (Fichiers)

Pour manipuler des fichiers système, Flutter combine **path_provider** (localise les dossiers système) et **dart:io** (classe `File` pour lire/écrire).

Méthode	Description
<code>getApplicationDocumentsDirectory()</code>	Dossier de l'app, persiste jusqu'à désinstallation
<code>getTemporaryDirectory()</code>	Cache système, peut être nettoyé par l'OS
<code>getApplicationSupportDirectory()</code>	Données de support (macOS/iOS uniquement)
<code>getDownloadsDirectory()</code>	Dossier Téléchargements (Desktop uniquement)

■ **Important** : `path_provider` ne supporte pas Flutter Web. Pour le web, utiliser `shared_preferences`, `IndexedDB` ou une API backend. Toujours utiliser `join()` du package `path` pour construire les chemins.

2.3 Partie C — SQLite avec sqflite

SQLite est un moteur de base de données relationnelle embarqué, stocké dans un unique fichier. Le package **sqflite** expose une API Dart asynchrone et thread-safe.

Méthode	Description
<code>openDatabase(path, ...)</code>	Ouvre ou crée la BDD. <code>onCreate</code> crée les tables au 1er lancement.
<code>db.insert(table, map)</code>	Insère un enregistrement. Retourne l'id généré.
<code>db.query(table, ...)</code>	Sélectionne des enregistrements → <code>List<Map<String, Object?>></code>
<code>db.update(table, map)</code>	Met à jour des enregistrements filtrés par <code>where/whereArgs</code> .
<code>db.delete(table, ...)</code>	Supprime des enregistrements filtrés par <code>where/whereArgs</code> .
<code>db.rawQuery(sql, args)</code>	Requête SQL brute avec placeholders — requêtes complexes.

■ **Sécurité SQL** : Toujours utiliser `whereArgs` pour éviter les injections SQL. JAMAIS : `where: 'id = ${dog.id}'`. TOUJOURS : `where: 'id = ?'`, `whereArgs: [dog.id]`.

3. TP2 — Application TaskVault (Intégration Complète)

TaskVault est un gestionnaire de tâches complet qui intègre les trois solutions de persistance dans une seule application Flutter cohérente. Chaque solution répond à un besoin concret différent.

3.1 Architecture et rôle de chaque solution

#	Solution	Ce qu'elle persiste	Pourquoi ce choix
A	shared_preferences	Prénom utilisateur + thème clair/sombre	Petites préférences, types primitifs
B	SQLite (sqflite)	Les tâches (id, titre, description, faite, date)	Données structurées, recherche, CRUD
C	Fichiers (path_provider)	Export des tâches en fichier .txt	Fichier texte, partage externe

3.2 Structure du projet Flutter

```
lib/  
  main.dart  
  models/  
    task.dart  
  services/  
    preferences_service.dart  
    database_service.dart  
    export_service.dart  
  screens/  
    onboarding_screen.dart  
    home_screen.dart  
    add_task_screen.dart
```

3.3 Dépendances (pubspec.yaml)

```
dependencies:  
  flutter:  
    sdk: flutter  
  shared_preferences: ^2.x.x  
  sqflite: ^2.x.x  
  path: ^1.x.x  
  path_provider: ^2.x.x
```

3.4 Description des composants

task.dart (Modèle)

- Classe Task avec : id (nullable avant insertion), title, description, isDone, createdAt.
- Méthode toMap() : convertit Task → Map pour SQLite (bool stocké en int 0/1).
- Factory Task.fromMap() : reconstruit un objet Task depuis une ligne SQLite.
- Méthode copyWith() : copie immuable avec champs modifiés (pattern pour toggle isDone).

PreferencesService (Partie A)

- Clés constantes privées pour éviter les fautes de frappe.
- getUsername() → String? : null au 1er lancement = déclencheur onboarding.
- setUsername(name) : sauvegarde le prénom après onboarding.

- `isDarkMode()` → bool : thème avec valeur par défaut false (clair).
- `clear()` : réinitialisation complète (déconnexion).

DatabaseService (Partie B)

- Pattern Singleton : `static Database? _db` — une seule instance partagée.
- Getter paresseux database : ouvre la BDD uniquement au premier appel.
- Table SQL : `id INTEGER PRIMARY KEY AUTOINCREMENT, title, description, is_done, created_at`.
- CRUD complet : `insertTask`, `getAllTasks`, `getPendingTasks`, `updateTask`, `deleteTask`.
- `getStats()` : retourne `{total, done, pending}` pour tableau de bord.

ExportService (Partie C)

- Localise le dossier avec `getApplicationDocumentsDirectory()`.
- Nom de fichier avec timestamp : `tasks_2025-01-15T10-30-00.txt`.
- `StringBuffer` pour construire le contenu (plus efficace que `String +=`).
- `listExports()` : liste tous les fichiers `.txt` existants.

Écrans Flutter

- `OnboardingScreen` : affiché si `getUserName() == null`. Sauvegarde le prénom.
- `HomeScreen` : liste réactive avec `Dismissible` (swipe → delete) et `Checkbox` (toggle).
- `AddTaskScreen` : formulaire création/édition (mode détecté via `task != null`).
- `main.dart` : theme adaptatif `ThemeMode.dark/light`, navigation conditionnelle onboarding.

3.5 Points clés et bonnes pratiques

Bonne pratique	Détail
<code>whereArgs</code> obligatoire	Prévient les injections SQL. Jamais d'interpolation directe.
Singleton pour la BDD	Évite les connexions multiples et l'overhead d'ouverture.
<code>copyWith()</code> immuable	Pattern dart pour modifier un objet sans muter l'original.
<code>StringBuffer</code> vs <code>+=</code>	$O(n)$ vs $O(n^2)$. Utiliser <code>StringBuffer</code> pour les concaténations multiples.
Clés const dans <code>PreferencesService</code>	Évite les typos, facilite le refactoring, encapsule le service.
<code>try/catch</code> sur <code>readAsString()</code>	Le fichier n'existe pas au 1er lancement — retourner valeur par défaut.
<code>WidgetsFlutterBinding.ensureInitialized()</code>	Requis avant tout accès async dans <code>main()</code> avant <code>runApp()</code> .

3.6 Flux de test de l'application

Étape	Action / Résultat
1er lancement	Écran Onboarding → saisir prénom → sauvegardé dans <code>shared_preferences</code>
Ajout de tâches	Bouton + → formulaire → tâche insérée en SQLite
Toggle tâche	Checkbox → <code>isDone</code> mis à jour → ligne barrée
Suppression	Swipe gauche (<code>Dismissible</code>) → DELETE SQLite
Export	Icône téléchargement → fichier <code>.txt</code> créé dans documents
Changement de thème	Icône soleil/lune → thème persisté dans <code>shared_preferences</code>
Relancement	Toutes les données persistent (prénom, thème, tâches)

4. BILAN ET EXTENSIONS

4.1 Récapitulatif des compétences acquises

- ✓ `shared_preferences` : stockage clé-valeur, types primitifs, `remove/clear`, mock pour tests.
- ✓ `path_provider` : `getApplicationDocumentsDirectory()`, `File`, `writeAsString/readAsString`.
- ✓ `sqlite` : `openDatabase`, `CREATE TABLE`, CRUD complet, `toMap/fromMap`, `whereArgs` sécurisé.
- ✓ Architecture Flutter en couches : `models` / `services` / `screens`.
- ✓ Patterns Dart avancés : Singleton, `copyWith()`, factory constructors, `StringBuffer`.
- ✓ Sécurité : prévention des injections SQL par `whereArgs` obligatoire.
- ✓ Choix de la bonne solution selon le type et le volume des données.

4.2 Extensions proposées (TP2)

- Priorité aux tâches (haute/normale/basse) → nouvelle colonne SQL.
- Recherche full-text → `db.rawQuery()` avec `LIKE`.
- Date d'échéance avec `DatePicker` → badge rouge si dépassée.
- Listes de tâches multiples (projets) → nouvelle table SQL avec FK.
- Partage de l'export → package `share_plus`, `Share.share(filePath)`.

4.3 Ressources officielles

Package	URL
<code>shared_preferences</code>	https://pub.dev/packages/shared_preferences
<code>path_provider</code>	https://pub.dev/packages/path_provider
<code>sqlite</code>	https://pub.dev/packages/sqlite
dart:io File API	https://api.dart.dev/dart-io/File-class.html
SQLite Tutorial	https://www.sqlitetutorial.net/